Water-ice clouds in a 250 K brown dwarf

This workshop follows the Virga cloud tutorials and the PICASO cloudy brown dwarf tutorial. Run the cells from top to bottom. We will calculate an H_2O cloud with Virga, compare clear and cloudy thermal spectra with PICASO, and plot two JWST spectra of WISE 0855. The final “Things to try” section links to tutorials for a full cloudy climate calculation.

What each program does: Virga uses water-ice optical constants and an atmospheric profile to calculate cloud optical properties. PICASO uses those cloud properties with gas opacity to calculate a spectrum. In the main exercise, the cloud changes but the 250 K temperature profile stays fixed. The linked climate tutorials show how to calculate a new temperature profile and cloud together.

Use an environment with the packages in `requirements.txt` and open Jupyter from this folder. The notebook uses the `picaso_reference` folder beside it, including a PICASO gas-opacity database for the spectra. Wavelengths are in μm, pressure is in bar, and temperature is in K.

1. Imports and input folders

Open this notebook from the workshop folder or the `solutions` folder. The cell below sets `picaso_refdata` to the local `picaso_reference` folder, which contains only the inputs used in this workshop. On the first run, it downloads PICASO’s gas-opacity database if it is missing. This is a large download (about 4.8 GB compressed and 7.3 GB unpacked), so allow time and disk space.

Code cell 3

```
from pathlib import Path
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import astropy.units as u
from astropy.io import fits

workshop_dir = Path.cwd()
if not (workshop_dir / 'picaso_reference').exists():
    workshop_dir = workshop_dir.parent
reference_dir = workshop_dir / 'picaso_reference'
os.environ['picaso_refdata'] = str(reference_dir)
os.environ['PYSYN_CDBS'] = str(workshop_dir / 'data' / 'stellar')

# PICASO's Quickstart downloads this separately from the Python packages.
opacity_file = reference_dir / 'opacities' / 'opacities_0.3_15_R15000.db'
if not opacity_file.exists():
    import picaso.data as data
    print('Downloading PICASO gas opacities; this may take a while...')
    data.get_data(category_download='resampled_opacity', target_download='default',
                  final_destination_dir=str(reference_dir / 'opacities'))
    if not opacity_file.exists():
        raise FileNotFoundError('The PICASO opacity database did not download.')

from picaso import justdoit as pdi
from virga import justdoit as vdi

bobcat_dir = reference_dir / 'sonora_grids' / 'bobcat'
virga_dir = reference_dir / 'virga'
data_dir = workshop_dir / 'data' / 'observations'
print('Reference data:', reference_dir)
```

```
WARNING: Failed to load Vega spectrum from None; Functionality involving Vega will be severely
limited: FileNotFoundError(2, 'No such file or directory') [stsynphot.spectrum]
Reference data: /Users/jjm6243/Documents/jwebbinar_prep/jwebbinar53/picaso_reference
```

2. Use the prepared water-ice cloud data

Water ice has a complex refractive index, n + ik, that changes with wavelength. The optical-constant table has already been converted into the model-ready `H2O.mieff` file supplied with this workshop. This file contains the Mie efficiencies Virga needs to calculate cloud optical depth, scattering, and asymmetry. We can use it directly, without repeating the file conversion during the workshop.

Code cell 5

```
mie_dir = virga_dir
mie_file = mie_dir / 'H2O.mieff'
if not mie_file.exists():
    raise FileNotFoundError(mie_file)
print('Water-ice Mie file:', mie_file.name)
```

```
Water-ice Mie file: H2O.mieff
```

Use the 250 K atmosphere for the cloud maps

Virga also needs a pressure–temperature profile, gravity, and a vertical mixing strength, K_{zz}. We use the 250 K Sonora Bobcat atmosphere and a constant illustrative K_{zz}=10^9 cm²/s. The cloud maps below use f_{sed}=4 as an example; Section 4 compares other values.

Code cell 7

```
teff = 250
gravity = 1000  # m/s²
radius_rj = 1.0
distance_pc = 2.2  # illustrative distance for the plotted flux
kzz = 1e9  # cm²/s; illustrative constant vertical mixing

columns = ['level', 'pressure', 'temperature', 'mu', 'H2', 'He', 'CH4',
           'CO', 'CO2', 'NH3', 'N2', 'H2O', 'TiO', 'VO', 'FeH', 'H2S']
atmosphere = pd.read_csv(bobcat_dir / 't250g1000nc_m0.0.cmp.gz',
                         sep=r'\s+', skiprows=1, names=columns)
atmosphere = atmosphere.drop(columns=['level', 'mu'])
pressure = atmosphere['pressure'].to_numpy()
cloud_atmosphere = atmosphere[['pressure', 'temperature']].copy()
cloud_atmosphere['kz'] = kzz

fsed_for_spectrum = 4
reference_cloud = vdi.Atmosphere(['H2O'], fsed=fsed_for_spectrum,
                                 mh=1, mmw=2.2, verbose=False)
reference_cloud.gravity(gravity=gravity, gravity_unit=u.m/u.s**2)
reference_cloud.ptk(df=cloud_atmosphere)
reference_output = reference_cloud.compute(directory=str(mie_dir))
print('Reference cloud: fsed =', fsed_for_spectrum)
```

```
Reference cloud: fsed = 4
```

Optical depth, scattering, and asymmetry

Virga's `all_optics` plot displays the cloud's optical depth per layer, single-scattering albedo, and asymmetry from the calculation above. These are **cloud optical properties**; the raw optical constants are the wavelength-dependent n and k used to make the Mie file.

Code cell 9

```
from virga import justplotit as vpi
from bokeh.io import output_notebook, show
from bokeh.resources import INLINE
```

```
output_notebook(resources=INLINE)
show(vpi.all_optics(reference_output))
```

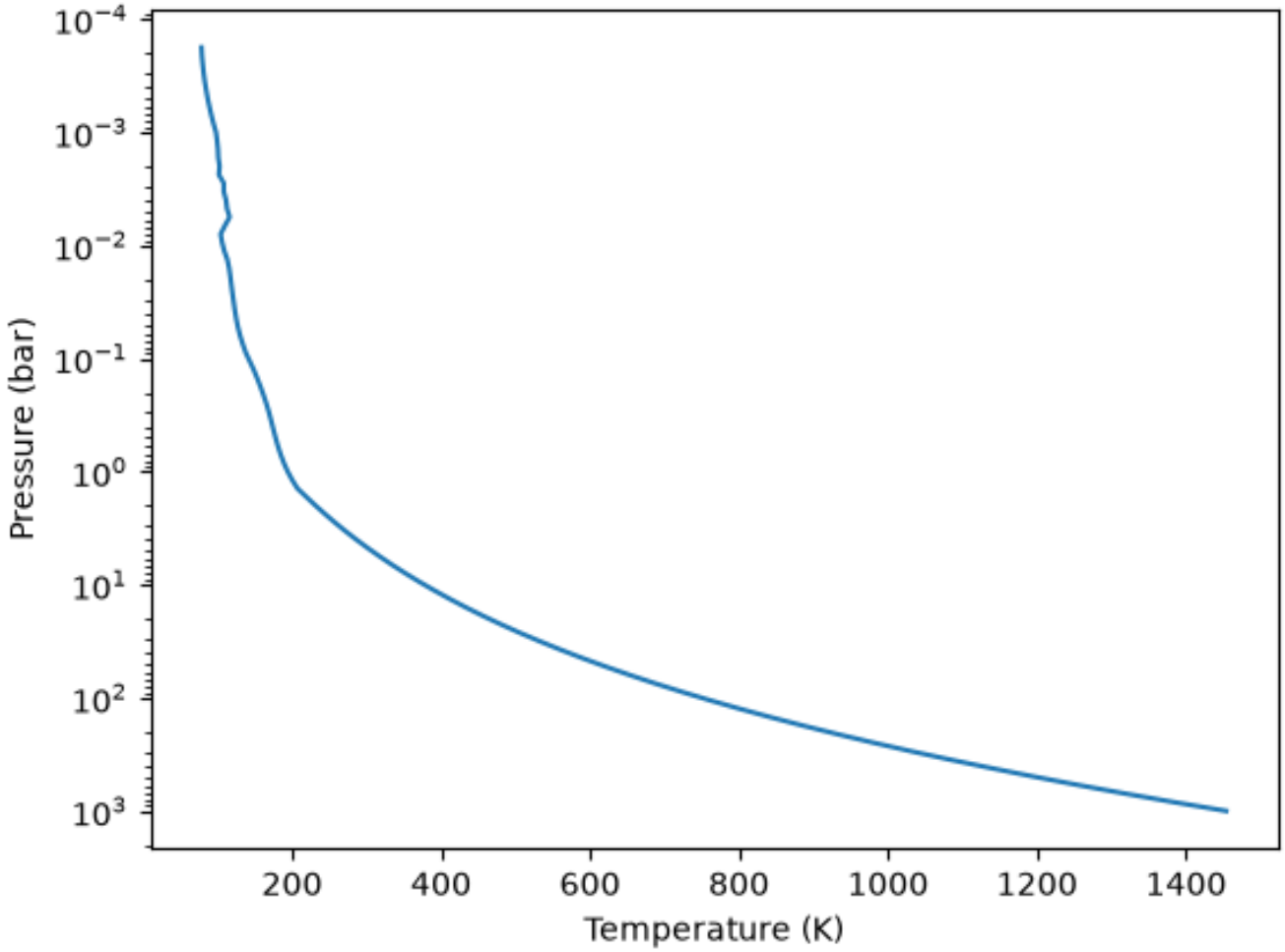
Interactive plot: open the solution notebook to explore this figure.

3. Inspect the 250 K atmosphere

The Sonora Bobcat file gives temperature and gas abundances at each pressure level. Plot the pressure–temperature profile before calculating a spectrum. Virga uses this profile to find where H_2O can condense; the main comparison will keep the profile fixed for both clear and cloudy cases.

Code cell 11

```
plt.semilogy(atmosphere['temperature'], pressure)
plt.gca().invert_yaxis()
plt.xlabel('Temperature (K)')
plt.ylabel('Pressure (bar)')
plt.show()
```



4. Compare sedimentation efficiencies

f_{sed} describes how efficiently cloud particles settle relative to upward mixing. **Smaller values generally give a more extended cloud; larger values give a more compact cloud.** Virga computes the cloud opacity and particle properties for each value. Here we compare f_{sed}=1, 4, and 8 on the same fixed atmosphere and with the same water-ice optical constants.

The plot shows **optical depth in each layer near 4 µm**, not cumulative optical depth. For the cloudy spectrum in Section 5, we use f_{sed}=4, the same value used for the maps above. The K_{zz} value and f_{sed} values are workshop examples, not fits to WISE 0855.

Code cell 13

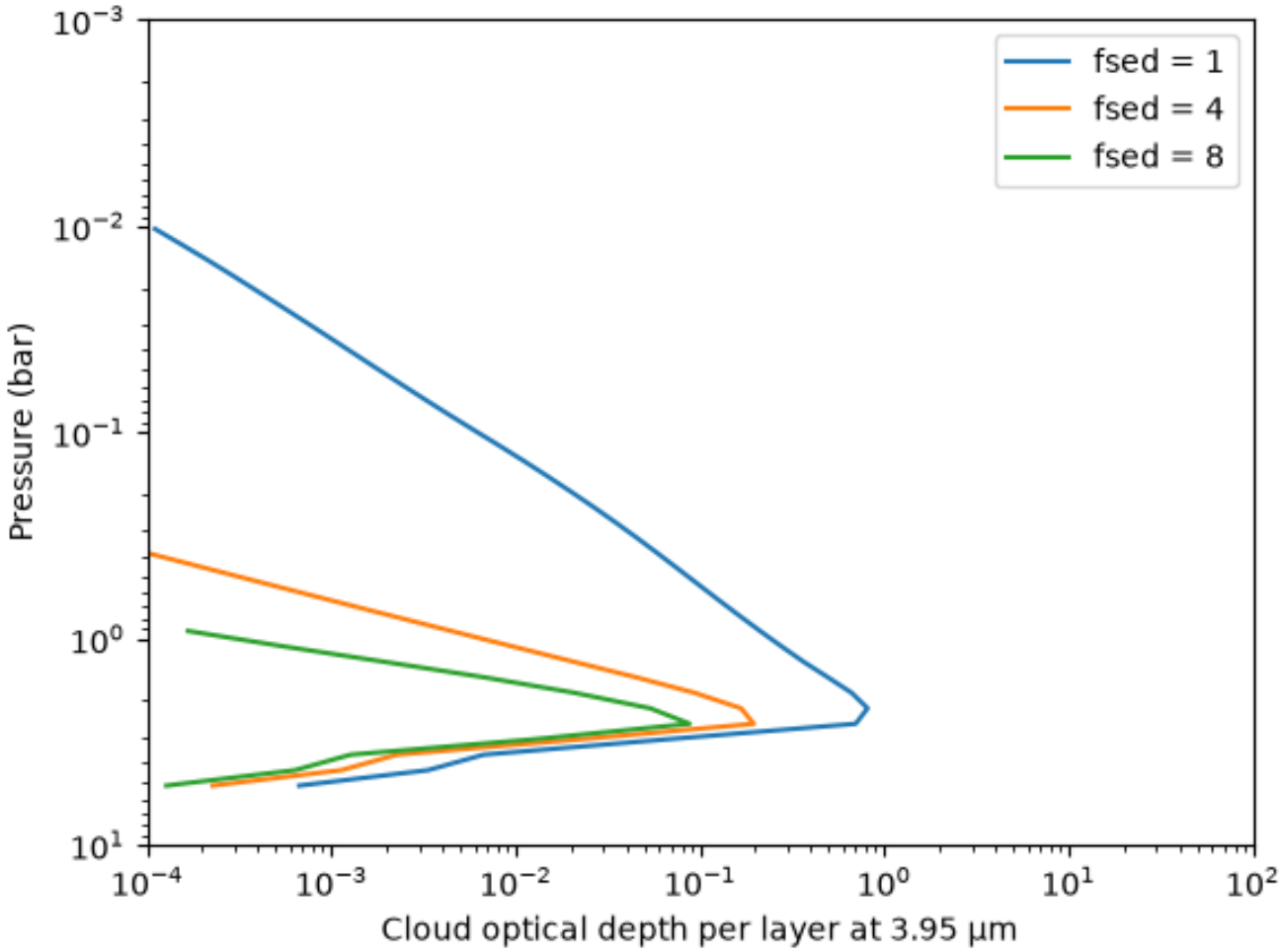
```
fsed_values = [1, 4, 8]
cloud_outputs = {fsed_for_spectrum: reference_output}

for value in fsed_values:
    if value == fsed_for_spectrum:
        continue
    cloud = vdi.Atmosphere(['H2O'], fsed=value, mh=1, mmw=2.2,
                           verbose=False)
    cloud.gravity(gravity=gravity, gravity_unit=u.m/u.s**2)
    cloud.ptk(df=cloud_atmosphere)
    cloud_outputs[value] = cloud.compute(directory=str(mie_dir))

cloud_wave = np.asarray(reference_output['wave'])
four_micron = np.argmin(np.abs(cloud_wave - 4.0))
for value in fsed_values:
    result = cloud_outputs[value]
    layer_tau = np.asarray(result['opd_per_layer'])[:, four_micron]
    layer_pressure = np.asarray(result['pressure'])
    visible = layer_tau >= 1e-4
    plt.plot(layer_tau[visible], layer_pressure[visible],
             label=f'fsed = {value:g}')

plt.xscale('log')
plt.yscale('log')
plt.xlim(1e-4, 1e2)
plt.ylim(10, 1e-3)
plt.xlabel(f'Cloud optical depth per layer at {cloud_wave[four_micron]:.2f} µm')
plt.ylabel('Pressure (bar)')
plt.legend()
plt.show()

cloud_result = cloud_outputs[fsed_for_spectrum]
print('Total cloud optical depth near 4 µm:',
      np.sum(cloud_result['opd_per_layer'][:, four_micron]))
cloud_df = vdi.picaso_format(cloud_result['opd_per_layer'],
                             cloud_result['single_scattering'],
                             cloud_result['asymmetry'],
                             pressure=cloud_result['pressure'],
                             wavenumber=1e4 / cloud_result['wave']).astype(float)
```




```
Total cloud optical depth near 4 μm: 0.56719258467122
```

5. Clear and cloudy thermal spectra

Brown dwarfs emit their own light, so we calculate **thermal emission**. First run PICASO with the 250 K atmosphere alone, then add Virga's f_{sed}=4 cloud to a second run. Since both use the same temperature and gas profile, differences in the curves come from the calculated cloud.

PICASO uses wavenumber (cm^{-1}), while the plot uses wavelength (μm): `wavelength = 1e4 / wavenumber`. PICASO returns surface flux. We convert it to flux density and scale it by `(radius / distance)**2` to compare with flux measured at Earth. The radius and distance here are illustrative, so this is a shape and scale comparison, not a fit.

We use K_{zz}=10^9 cm^2/s and f_{sed}=4 for a moderate cloud effect with the bundled water-ice optical properties. This is a visual workshop comparison, not a fitted model.

Code cell 15

```
opa = pdi.opannection(wave_range=[2, 15], filename_db=str(opacity_file))

clear = pdi.inputs(calculation='browndwarf')
clear.gravity(gravity=gravity, gravity_unit=u.m/u.s**2)
clear.atmosphere(df=atmosphere)
clear_spectrum = clear.spectrum(opa, calculation='thermal', full_output=True)

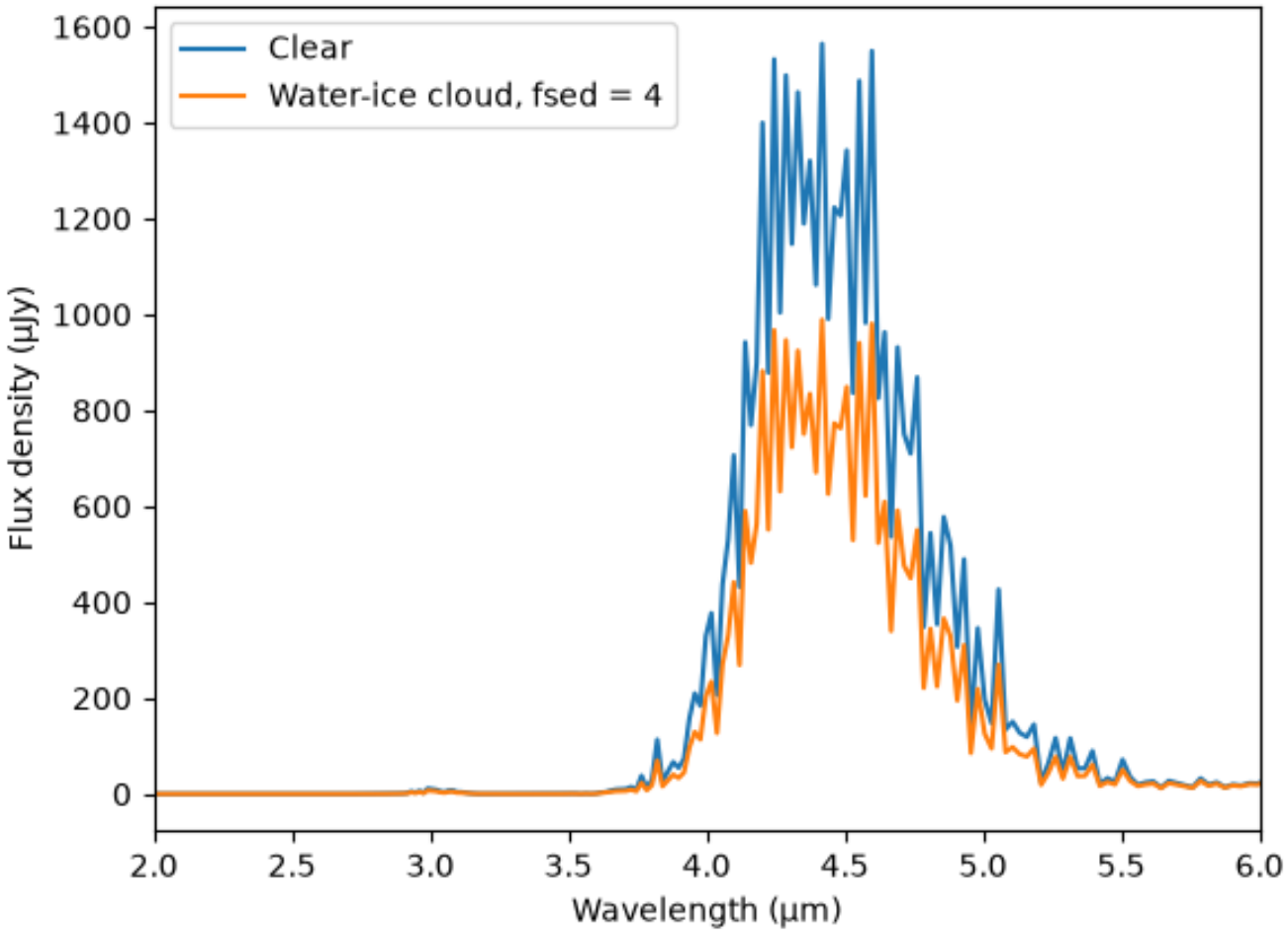
cloudy = pdi.inputs(calculation='browndwarf')
```

```
cloudy.gravity(gravity=gravity, gravity_unit=u.m/u.s**2)
cloudy.atmosphere(df=atmosphere)
cloudy.clouds(df=cloud_df)
cloudy_spectrum = cloudy.spectrum(opa, calculation='thermal', full_output=True)

# Convert each surface spectrum to flux per Hz, then regrid for display.
clear_fnu = pdi.convert_flux_units(clear_spectrum['wavenumber'],
                                   clear_spectrum['thermal'],
                                   to_f_unit='erg*cm^(-2)*s^(-1)*Hz^(-1)')
cloudy_fnu = pdi.convert_flux_units(cloudy_spectrum['wavenumber'],
                                    cloudy_spectrum['thermal'],
                                    to_f_unit='erg*cm^(-2)*s^(-1)*Hz^(-1)')
clear_wno, clear_fnu = pdi.mean_regrid(clear_spectrum['wavenumber'], clear_fnu, R=200)
cloudy_wno, cloudy_fnu = pdi.mean_regrid(cloudy_spectrum['wavenumber'], cloudy_fnu, R=200)

scale = ((radius_rj * u.R_jup) / (distance_pc * u.pc)).to_value(u.dimensionless_unscaled)**2
clear_wave = 1e4 / clear_wno
cloudy_wave = 1e4 / cloudy_wno
clear_ujy = clear_fnu * scale / 1e-23 * 1e6
cloudy_ujy = cloudy_fnu * scale / 1e-23 * 1e6

plt.plot(clear_wave, clear_ujy, label='Clear')
plt.plot(cloudy_wave, cloudy_ujy, label=f'Water-ice cloud, fsed = {fsed_for_spectrum}')
plt.xlim(2, 6)
plt.xlabel('Wavelength (μm)')
plt.ylabel('Flux density (μJy)')
plt.legend()
plt.show()
```



6. Read the two JWST observations

The FITS files in `data/observations/` were downloaded from MAST as examples: Program 1230 (PI: Catarina Alves de Oliveira) has one G395M x1d spectrum, and Program 2327 (PI: Andrew Skemer) has 44 G395M integrations in an `x1dints` file. We take the **median across those integrations at each wavelength** to make one comparison spectrum. The shaded 16th–84th percentile range in the next plot shows the spread of the integrations, not the uncertainty on the median.

Both files give wavelength in μm and flux in Jy. We multiply by one million to plot μJy. We keep points with `DQ == 0`, a finite flux, and a positive flux error. An additional `|flux| < 0.01` Jy cut removes a severe unflagged artifact near 5.166 μm in program 2327. This is a display choice for the workshop; inspect the reduction before fitting models.

Code cell 17

```
file_1230 = data_dir / 'jw01230-o007_t001-s000000001_nirspec_f290lp-g395m-s200a1-subs200a1_x1d.fits'
file_2327 = data_dir / 'jw02327-o001_t001_nirspec_f290lp-g395m-s1600a1-sub2048_x1dints.fits'

with fits.open(file_1230) as hdul:
    spectrum_1230 = hdul['EXTRACT1D'].data
    wave_1230 = np.asarray(spectrum_1230['WAVELENGTH'], dtype=float)
    flux_1230 = np.asarray(spectrum_1230['FLUX'], dtype=float)
    error_1230 = np.asarray(spectrum_1230['FLUX_ERROR'], dtype=float)
    dq_1230 = spectrum_1230['DQ']

good_1230 = ((dq_1230 == 0) & np.isfinite(flux_1230)
             & (error_1230 > 0) & (np.abs(flux_1230) < 0.01))

with fits.open(file_2327) as hdul:
    integrations = [hdu.data for hdu in hdul if hdu.name == 'EXTRACT1D']
    wave_2327 = np.vstack([row['WAVELENGTH'] for row in integrations])[0]
    flux_2327 = np.vstack([row['FLUX'] for row in integrations]).astype(float)
    error_2327 = np.vstack([row['FLUX_ERROR'] for row in integrations]).astype(float)
    dq_2327 = np.vstack([row['DQ'] for row in integrations])

good_2327 = ((dq_2327 == 0) & np.isfinite(flux_2327)
             & (error_2327 > 0) & (np.abs(flux_2327) < 0.01))
flux_2327[~good_2327] = np.nan

# Some edge wavelengths have no good integrations; omit those columns.
has_data = np.sum(good_2327, axis=0) >= len(flux_2327) / 2
median_2327 = np.nanmedian(flux_2327[:, has_data], axis=0)
low_2327 = np.nanpercentile(flux_2327[:, has_data], 16, axis=0)
high_2327 = np.nanpercentile(flux_2327[:, has_data], 84, axis=0)
wave_2327 = wave_2327[has_data]

print('Program 1230:', good_1230.sum(), 'points')
print('Program 2327:', len(flux_2327), 'integrations,', len(wave_2327), 'wavelengths')
```

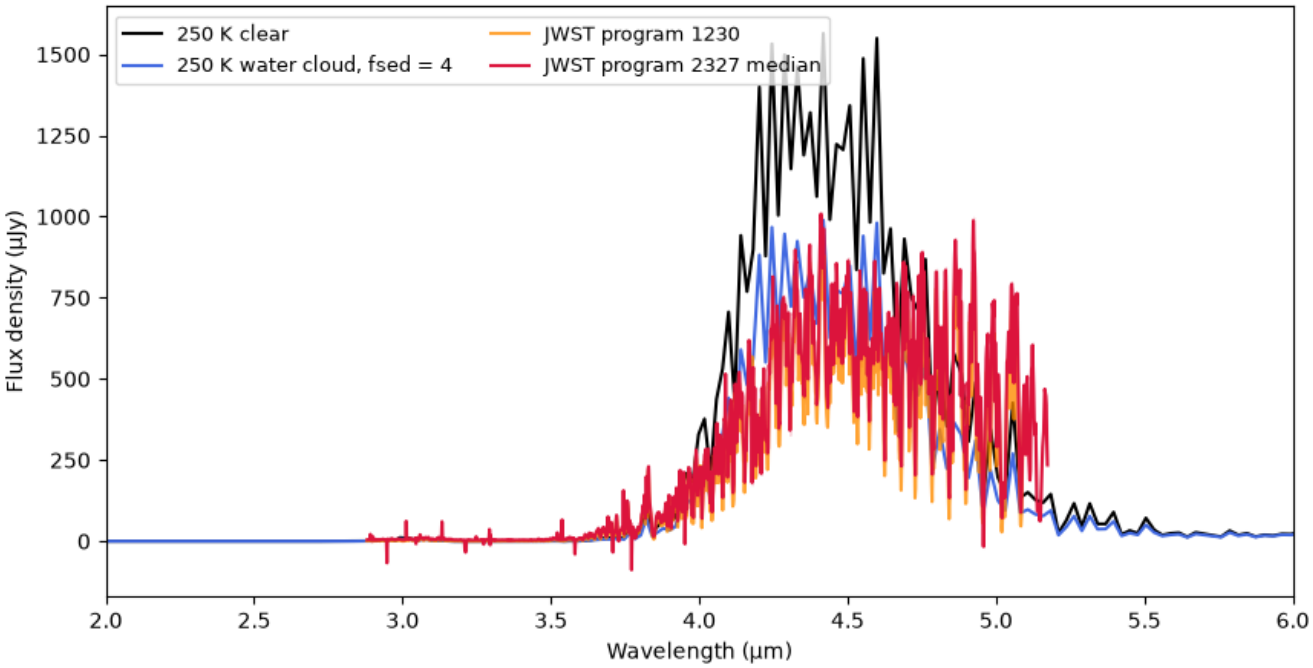
```
Program 1230: 1233 points
Program 2327: 44 integrations, 1275 wavelengths
```

Compare models and data

This plot uses the same 2–6 μm wavelength range as the clear and cloudy spectrum plot above. The observed spectra stay on their native wavelength grids. Both model spectra are shown at about R = 200 for a readable figure. The model radius, distance, mixing, and sedimentation assumptions are illustrative; these curves have not been fitted or convolved with the instrument response.

Code cell 19

```
plt.figure(figsize=(10, 5))
plt.plot(clear_wave, clear_ujy, color='black', label='250 K clear')
plt.plot(cloudy_wave, cloudy_ujy, color='royalblue',
         label=f'250 K water cloud, fsed = {fsed_for_spectrum}')
plt.plot(wave_1230[good_1230], flux_1230[good_1230] * 1e6,
         color='darkorange', alpha=0.8, label='JWST program 1230')
plt.plot(wave_2327, median_2327 * 1e6,
         color='crimson', label='JWST program 2327 median')
plt.fill_between(wave_2327, low_2327 * 1e6, high_2327 * 1e6,
                 color='crimson', alpha=0.15)
plt.xlim(2, 6)
plt.xlabel('Wavelength (μm)')
plt.ylabel('Flux density (μJy)')
plt.legend(ncol=2, fontsize=9)
plt.show()
```



7. Things to try

- Change f_{sed} or K_{zz}, one at a time. How do the 4 μm optical-depth profile and cloudy spectrum change?
- Look at notebook 01 for an example of turning an optical-constant table into a `.mieff` file.
- Compare the two JWST observations. For a quantitative fit, inspect their reductions and account for the instrument response.

Going further: self-consistent cloud models

In this workshop, we start from a fixed 250 K Sonora temperature–pressure profile. Virga calculates a water cloud on that profile, and PICASO calculates the resulting spectrum. The cloud does not change the temperature profile in this short calculation.

A self-consistent cloudy climate model takes the next step: it iterates the temperature structure and cloud properties together until the atmosphere satisfies radiative–convective energy balance. Changing the cloud can change the temperature, which can then change the cloud. These calculations take longer and their convergence can depend on the starting convective-boundary guess, so they are better explored after the workshop.

PICASO tutorials to explore when you are ready:

1. Clear brown dwarf climate model — learn the climate setup without clouds. 2. Brown dwarf climate model with Virga clouds — couple water clouds to the climate calculation. 3. Common climate-model issues — check convergence and troubleshoot starting guesses.

The full PICASO tutorial index has more examples. Cite the Sonora Bobcat models and the sources of the JWST data in scientific work.